The Iron Rule Checklist: Structured Specification Elicitation Reduces False-Pass Rates from 82% to 3.5% in LLM-Generated Lean 4 Specifications

Tang Meng
Independent Researcher, Shanghai, China
With Apart Research

Research conducted at the Secure Program Synthesis Hackathon, May 2026.

Abstract

When users describe programming tasks in natural language and an LLM generates Lean 4 formal specifications, how often does the result silently encode the wrong behavior? We measure the **false-pass rate**—the fraction of objectively insufficient descriptions that produce specifications passing all decide-based verification tests—across 243 natural-language descriptions (81 VERINA benchmark problems × 3 communication personas). At baseline, 82.4% of insufficient descriptions produce false-passing specs. Free-form LLM questioning reduces this to 16.4%. The *Iron Rule Checklist*, a structured 7-category protocol, reduces it to 3.5%—with **zero regressions** from the intermediate condition and 4 of 8 issue types reaching **100% detection**. This is the first controlled comparison of specification elicitation methods for formal verification. The 6 remaining misses decompose into three structurally distinct categories—NL ambiguity, example inadequacy, and formal spec completeness gaps—defining the method's theoretical ceiling and a concrete research agenda. Without structured elicitation, "formally verified" is a label on the process, not on the user's intent.

1. Introduction

Consider a user who asks an LLM to formalize this requirement: *"Sort the list from smallest to largest."* The LLM generates a Lean 4 postcondition: `result.Sorted (· ≤ ·)`. Lean's kernel verifies it against test cases. Everything passes. But the specification is wrong: it accepts *any* non-decreasing list, not just a permutation of the input. The empty list is a valid sort of [4, 2, 1, 3]. The user sees "Lean verified ✓" and ships the code.

This is not a contrived example—it is case adv17-A from our dataset. When we gave Claude free rein to ask clarifying questions (C2), it asked about sort stability, not about element preservation—and missed the gap. When we gave Claude the Iron Rule Checklist (C3), category V ("Output Constraints") generated the question: *"Must the output be a permutation of the input?"* The gap was caught. This pattern—obvious to a human in hindsight, invisible to an LLM without structured guidance—repeats across **82.4%** of our 226 test cases.

We submit to **Track 1 (Specification Elicitation)**. A growing consensus in Secure Program Synthesis (SPS) holds that specification is the binding bottleneck: "specs are expensive, proofs are cheap"[1,2]. Lahiri[3] identifies the "intent gap" as the central reliability bottleneck. Yet no study has quantified how large this gap is, nor evaluated interventions. Our contributions:

1. Quantitative diagnosis. 82.4% of objectively insufficient NL descriptions produce false-passing Lean 4 specs—"verified" code that does not satisfy actual requirements.

2. The Iron Rule Checklist. A 7-category elicitation protocol that reduces the false-pass rate to 3.5%, with zero regressions from free-form Q&A; (16.4%)—the first controlled comparison of specification elicitation methods for formal verification.

3. Theoretical ceiling. The 6 remaining misses decompose into three structural categories, each mapping to a distinct research direction: checklist extension, example verification, and formal output-constraint auditing.

4. Reusable framework. The persona × checklist × three-condition design is applicable to any NL-to-formal-spec pipeline.

2. Related Work

The specification bottleneck. Von Hippel et al.[4] identify specification as SPS's binding constraint. Dougherty[2] proposes agentic intermediate representations (IRs) for the elicitation-validation loop—the process by which informal intent becomes formal spec and is checked for faithfulness. Lahiri[3] surveys intent formalization approaches, identifying spec validation as the core unsolved problem. Our work provides the first quantitative evidence for this loop's importance: without it, 82% of specs encode the wrong behavior.

Verification gaps. Dougherty and von Hippel[5] catalog five ways formal proofs mislead, including semantic drift (spec doesn't capture intent) and accidental triviality (spec is vacuously true). CLEVER[6] reports spec certification rates below 2%. VERINA[7] uses decide-based verification—Lean's kernel-level decision procedures on concrete test cases. This is computationally sound but covers only the tests provided. When complete proofs are unavailable—the common case[8,9]—this incomplete coverage is the norm, and we quantify the resulting vulnerability.

3. Methods

3.1 Dataset

81 advanced problems from VERINA[7], each with a precise task description, Lean 4 template, and soundness/completeness test cases. Three personas write independent NL descriptions: **A** (goal-only), **B** (example-driven), **C** (process-oriented). All write colloquially without constraints or edge cases. 81 × 3 = 243 descriptions. Each is classified as **SHOULD NOT PASS** (226: specific gap + multiple interpretations + distinguishing input) or **SHOULD PASS** (17). Gaps are tagged with 8 issue types (Table 1).

Table 1. *Issue types in 226 SHOULD NOT PASS descriptions, with baseline false-pass rate.*

Type	Description	N	C1 FP%
A — Ambiguity	Lexical ambiguity ("section" = contiguous?)	53	81%
Pre — Precondition	Unstated input assumption	41	82%
D — Definition	Standard term with boundary gap	39	91%
E — Edge case	Unspecified boundary behavior	37	71%
O — Output	Output format/ordering omission	21	95%
T — Tiebreaker	Missing disambiguation rule	17	79%
P — Preprocessing	Unstated normalization step	14	77%
R — Representation	Ambiguous encoding ("digit" base?)	4	75%

3.2 The Iron Rule Checklist

A structured 7-category protocol. No item may be skipped: **I.** Input Space Enumeration (boundary values); **II.** Term Definition Boundaries (standard definitions vs. legal inputs); **III.** Representation Ambiguity (unstated encoding); **IV.** Multiple Valid Answers (disambiguation rules); **V.** Output Constraints (format/ordering/permutation); **VI.** Preconditions (input assumptions); **VII.** Example Audit (Persona B: can multiple rules explain the example?). The sorting example from §1 is a category V catch.

3.3 Three-Condition Experiment

All 243 descriptions pass through three conditions, each using a fresh Claude instance with zero cross-problem context. **C1 (Baseline):** NL + input/output types fed directly to the LLM for specification generation. Function bodies replaced with sorry; solution auxiliaries stripped; decidable predicates required (10M-heartbeat limit). 207/243 (85.2%) compile; 36 failures are all AI code quality. **C2 (Free Q&A;):** The LLM generates clarifying questions (one round, unlimited count; 712 total questions across 243 cases). Answers drawn from VERINA ground truth—brief, honest, no volunteered information. **C3 (Checklist-Guided Q&A;):** Identical protocol, but

the LLM receives the Iron Rule Checklist as a structured skill. **Judgment:** Each of 226 SNP cases individually audited with full question transcripts. Verdicts: Hit/Fix/Partial/Miss. 18 corrections applied across C2 audit rounds.

4. Results

Figure 1. *(a) False-pass rate drops monotonically across conditions. (b) Per-issue-type improvement: orange dots = C2 hit rate; green dots = C3. Four types reach 100%.*

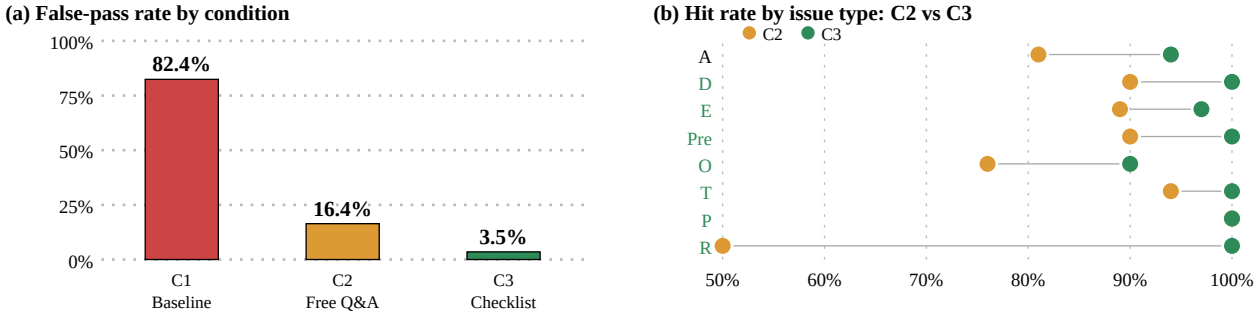

Table 2. *Three-condition comparison on 226 SHOULD NOT PASS descriptions.*

	C1 (Baseline)	C2 (Free Q&A)	C3 (Checklist)
Hit (notice issue)	—	196 (86.7%)	220 (97.3%)
Fix (resolve issue)	—	189 (83.6%)	218 (96.5%)
Partial	—	7 (3.1%)	2 (0.9%)
Miss	—	30 (13.3%)	6 (2.7%)
False-pass rate	82.4%	16.4%	3.5%

The false-pass rate drops monotonically: 82.4% → 16.4% → 3.5%. Free Q&A; is a substantial improvement; the checklist nearly eliminates false passes entirely. All three personas improve (A: 89→98% hit; B: 86→96%; C: 85→99%), with process-oriented descriptions showing the largest gain—consistent with the checklist targeting unstated procedural assumptions. SHOULD PASS controls remain stable (14/14 C1, 17/17 C2): no false negatives.

4.1 Two Surprising Findings

Zero regressions. The checklist is not merely better on average—it is a *strict superset* of free Q&A;'s coverage. Every case free Q&A; resolves, the checklist also resolves (189/189). Of 30 C2 misses, C3 catches 24. Of 7 C2 partials, C3 fully resolves 5. No case goes backward. This zero-regression property was not designed into the checklist—it emerged from the structured enumeration's ability to subsume free-form questioning.

Four issue types reach 100%. Definition (D), Precondition (Pre), Tiebreaker (T), and Representation (R) are fully eliminated by the checklist (Table 3, Figure 1b). These correspond to checklist categories II, VI, IV, and III—"closed" gap types with finite, enumerable failure modes. Structured enumeration is sufficient. Output constraints and Ambiguity resist full elimination (90%, 94%), requiring deeper semantic analysis.

Table 3. *Per-issue-type hit rates. Bold = 100% detection.*

Type	N	C2 Hit%	C3 Hit%	Δ
D (Definition)	39	90%	**100%**	+10
Pre (Precondition)	41	90%	**100%**	+10
T (Tiebreaker)	17	94%	**100%**	+6
R (Representation)	4	50%	**100%**	+50

Type	N	C2 Hit%	C3 Hit%	Δ
E (Edge case)	37	89%	97%	+8
A (Ambiguity)	53	81%	94%	+13
O (Output)	21	76%	90%	+14
P (Preprocessing)	14	100%	100%	—

4.2 The 6 Remaining Misses: A Research Agenda

The misses decompose into three structurally distinct categories. Each defines a different research direction:

Category 1: NL ambiguity (2 cases). The NL contains an ambiguous core operation word the checklist does not probe. In adv40-A, "top value" could mean maximum, first, or last. *Research direction:* extend the checklist with a "verify core operation word" category—directly addressable.

Category 2: Example inadequacy (2 cases). The NL is clear in English, but the example cannot distinguish the interpretations (e.g., all distinct values make > and ≥ equivalent). *Research direction:* generate distinguishing test cases automatically—partially addressable with example-audit tooling.

Category 3: Formal spec completeness (2 cases). "Biggest of three" has no NL ambiguity, but result ≥ a ∧ result ≥ b ∧ result ≥ c admits result = 999. The missing membership constraint is a spec-level, not NL-level, problem. **Question-based elicitation cannot reach this class.** *Research direction:* formal output-constraint auditing—the kind of round-trip check that Claimcheck[10] and mutation testing approaches[2] aim to provide.

Excluding Category 3 (which is outside any elicitation method's scope), the effective miss rate on NL-level issues is 4/226 = **1.8%**.

5. Discussion and Limitations

Verification ≠ validation. Our 82.4% baseline gives empirical content to a distinction the field has discussed abstractly: decide-based verification confirms internal consistency between spec and tests, but cannot detect semantic drift between user intent and the LLM's interpretation. The result is verification theatre—the user sees "Lean verified ✓" while the spec encodes the wrong behavior. This is the exact failure mode that makes AI-generated "verified" code dangerous: the checkmark creates false confidence that formal methods were supposed to prevent.

Layered defense. The per-issue-type results reveal a defense structure: closed gap types (D, Pre, T, R) are fully eliminated by enumeration; open gap types (O, A) require complementary tools (mutation testing, semantic comparison); formal spec gaps (Category 3) require output-constraint auditing. The Iron Rule Checklist handles layer one; layers two and three define the remaining research agenda. The checklist's categories map directly to the intermediate representations in Dougherty's Agentic IRs framework[2]: each category is a semantic hop in the elicitation-validation loop.

Limitations. (1) We use a single LLM (Claude) for both question generation and spec generation; the checklist's effectiveness may interact with model capability. (2) VERINA consists of algorithm problems—real-world specification tasks differ structurally. (3) NL descriptions are researcher-constructed. (4) We measure gap *detection*, not end-to-end spec correction. Next steps: close the loop by re-generating specs from augmented NLs, repeat with multiple LLMs, and evaluate on non-algorithmic tasks.

6. Conclusion

We present the first quantitative measurement of false-pass rates in LLM-generated formal specifications and the first controlled comparison of specification elicitation methods. The Iron Rule Checklist reduces false passes from 82.4% to 3.5% with zero regressions, reaches 100% detection on 4 of 8 issue types, and defines a precise theoretical ceiling through the 6 remaining misses. The bottom line: without structured elicitation, "formally verified" is a label that attaches to the process, not to the user's intent.

Code and Data

github.com/tomnovember/sps-iron-rule-checklist — 243 NL descriptions with ground truth, C2/C3 judgment records with full question transcripts, the Iron Rule Checklist, and all pipeline scripts.

Author Contributions

T.M. designed the experiment, constructed NL descriptions, developed the Iron Rule Checklist, executed the three-condition experiment, audited all 226 judgments, and wrote the paper.

References

[1] von Hippel, M., Henniger, S., Dougherty, Q., Miyazono, E. (2026). How to Solve Secure Program Synthesis. LessWrong.

[2] Dougherty, Q., von Hippel, M. et al. (2026). Tractable Problems in AI Security via Formal Methods. tractable.for-all.dev.

[3] Lahiri, S. (2026). Intent Formalization. arXiv:2603.17150.

[4] von Hippel, M. (2026). The Scalable Formal Oversight Research Program. LessWrong.

[5] Dougherty, Q., von Hippel, M. (2026). Lies, Damned Lies, and Proofs: Formal Methods are not Slopless. LessWrong.

[6] Yang, K. et al. (2025). CLEVER: A Benchmark for Closed-Loop Verifiable Code Generation. NeurIPS 2025.

[7] Sun, C. et al. (2025). VERINA: Benchmarking LLMs on Verified Program Reasoning in Lean 4. ICLR 2026. arXiv:2505.23135.

[8] SorryDB (2026). sorrydb.com. Tracking sorry usage across Lean 4 projects.

[9] Lean Community (2026). native_decide trust model. Lean FAQ.

[10] Henniger, S., Amin, N. (2026). Claimcheck: Detecting Intent Drift via Round-Trip Informalization. Harvard (SPS fellowship project).

LLM Usage Statement

Claude (Anthropic) was used as both the experimental subject (generating specifications and clarifying questions in all three conditions) and as an analysis assistant. All 226 case verdicts were independently verified by the author against full question transcripts. The Iron Rule Checklist was developed iteratively through human review of failure cases. Claude assisted in drafting this report; all claims were verified.

Appendix: Limitations and Dual-Use Considerations

Dual-use. Our work identifies weaknesses in NL-to-spec pipelines. The defensive value (the checklist and quantitative failure-mode understanding) substantially outweighs the risk; the failure modes are straightforward to discover independently. **Scope.** Results are specific to VERINA (algorithmic problems), decide-based verification, and one LLM (Claude). The relative ordering (baseline >> free Q&A; >> checklist), the zero-regression property, and the structural ceiling analysis are the robust findings; absolute rates are context-dependent.